Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

23/07/2024

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 17 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct exam delivery.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider a dam equipped with a Supervisory Control And Data Acquisition (SCADA) system, an industrial control system used to monitor and control the dam's operations. The SCADA system collects data from various sensors (e.g., water level, flow rate, temperature) and sends it to a central control room, where operators can monitor the dam's status and make adjustments as needed. The system also controls the opening and closing of gates and valves, as well as the operation of pumps and turbines. The SCADA system is connected to the Internet for remote monitoring and control, and it uses a combination of wired and wireless communication protocols.

[1.5 points] 1. Identify and describe two relevant assets at risk in such a scenario, along with their associated threats and threat agents. All elements must be motivated.

Asset	Threat	Threat Agent
Dam Infrastructure (Physical and Cyber)	**Threat:** Unauthorized remote access or manipulation of the SCADA system could lead to the opening or closing of gates/valves at the wrong time, causing flooding or water shortages.	**Threat Agent:** Malicious hackers or state-sponsored actors seeking to cause physical damage or disrupt water supplies.
Environmental Impact	**Threat:** A cyberattack could lead to the release of large amounts of water, causing significant environmental damage to downstream ecosystems and communities.	**Threat Agent:** Eco-terrorists or individuals with a grudge against the dam's operators.

[1 point] 2. Suggest at least two potential attack surfaces for the infrastructure under analysis.

Attack surface	Motivation
The SCADA system itself	The SCADA system is a complex network of interconnected devices and software, making it a prime target for attackers. Vulnerabilities in the system could be exploited to gain unauthorized access, disrupt operations, or cause physical damage.
Communication networks	The SCADA system relies on communication networks to transmit data between sensors, controllers, and the central control room. These networks could be intercepted or disrupted, leading to a loss of control or data breaches.
Physical access to the dam	If attackers gain physical access to the dam, they could potentially tamper with equipment, bypass security measures, or cause physical

	damage.
Remote access points	Remote access points, such as those used for maintenance or monitoring, could be exploited by attackers to gain unauthorized access to the SCADA system.
Human operators	Human operators are a potential attack surface, as they could be tricked into divulging sensitive information or performing actions that compromise the system's security.

[1.5 points] 3. A friend of yours suggests using a custom challenge and response protocol to authenticate devices for "remote monitoring and control" that works as follows: devices involved in the communication have to share the SHA3 of a preshared secret, securely stored in each device. If the hashes match on both endpoints, the authentication is successful. Highlight the weaknesses of the proposed protocol and suggest a modification to make it working.

Weaknesses of the Proposed Protocol:

1. **Lack of Freshness**:
    - The use of a static hash for authentication means that if an attacker intercepts the hash value during transmission, they can replay it to gain unauthorized access. This is known as a replay attack.
2. **Exposure to Man-in-the-Middle Attacks**:
    - Without a mechanism to ensure the integrity and confidentiality of the transmitted hash, an attacker could intercept and modify the communication, potentially replacing the legitimate hash with their own.
3. **Preshared Secret Management**:
    - The protocol requires secure storage and management of the preshared secret on each device. If the secret is compromised on any device, the entire authentication scheme is vulnerable.
4. **No Mutual Authentication**:
    - The protocol as described only verifies the identity of the device sending the hash, not the receiver. This could allow an attacker to impersonate the control system to the device.

Suggested Modification to Improve the Protocol:
To address these weaknesses, the protocol can be enhanced by incorporating nonces (random values) and mutual authentication. Here's a modified version:

1. **Introduce Nonces for Freshness**:
    - Both the device and the control system generate random nonces for each authentication session to prevent replay attacks.
2. **Mutual Authentication**:
    - Both parties authenticate each other, ensuring that the communication is between legitimate devices and the control system.
3. **Use HMAC with a Secure Hash Function**:
    - Use HMAC (Hash-based Message Authentication Code) instead of a plain hash function to include the nonce in the authentication process.

Exercise 2 - Introduction to Cryptography (6 points)

Consider a file encryption system acting individually on each file present in a (hierarchical) filesystem, such as the one on common PCs and smartphones. The file encryption system encrypts each file employing AES-256 in Counter (CTR) mode. The initial value of the counter is obtained by taking the first 16 characters of the filename, each one represented as an 8 bit value, and concatenating them. The AES-256 key is derived from a user supplied password. The key management procedures are properly implemented.

[3 points] 1. Consider the password choice to be appropriate for this question. Is the described file encryption scheme providing confidentiality against a (computationally bound) attacker who has only access to the encrypted files alone? If this is not the case, describe the information leakage taking place and propose a way to fix the scheme, if this is the case, argue why no information is leaked, optionally providing the sketch of a proof.

The proposed encryption scheme is not secure. Indeed, the system is employing a counter-mode encryption scheme with a poor nonce choice: the first 16 characters of the filename may indeed be repeated across different files, and not used only once. In particular whenever two files share the first 16 characters of the filename, they will be encrypted adding the same keystream to the plaintext. An attacker can compute their bitwise difference (xor) and obtain the unencrypted difference between the corresponding plaintexts, which is a clear information leakage.

[3 points] 2. Consider the problem of choosing passwords for the aforementioned scheme, determine how many words must be employed in a *diceware* approach, assuming a dictionary with 2^16 words, to obtain a security margin equivalent to the selection of a uniformly random AES-256 key.

A uniformly randomly drawn AES-256 key is sampled from a r.v. with 256 bit entropy, defined over the set of the 2^256 possible keys. Employing a diceware approach, each (uniform) sample from the 2^16 words dictionary is sampling from a r.v. with 16b of entropy. Therefore, 16 independently sampled words are required to match the information content of a random 256 bit key.

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <unistd.h>
4. #include <stdlib.h>
5.
6. typedef struct{
7.     char name[4];
8.     char description[36];
9.     int hack_index;
10. } hackémon;
11.
12. void add_hackémon(){
13.     hackémon h;
14.
15.     // TODO: allow the user to enter multiple hackémons
16.     h.hack_index = 1;
17.
18.     while (h.hack_index > 0){
19.         // TODO: allow the user to enter a custom description
20.         strcpy(h.description, "hakachu is the best hackémon");
21.
22.         puts("Enter the hackémon name:");
23.         scanf("%112s", h.name);
24.
25.         puts("New hackémon added!");
26.         printf(h.description);
27.
28.         // TODO: add the hackémon to the hackedex
29.         h.hack_index--;
30.     }
31. }
32.
33. int main(){
34.     clearenv();
35.     add_hackémon();
36.     return 0;
37. }
```

*The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Motivate and Modify the line to solve the detected vulnerability**
Buffer Overflow	**[0.25] Lines 23;**	**See Slides [0.25]**
Format String	**[0.25] Lines 26;**	**If combined with BOF… See Slides [0.25]**

[2 points] 2. Focus only on the stack-based buffer overflow(s) you found. Write an exploit for this vulnerability that **must execute a very sophisticated shellcode** composed of **60** bytes.

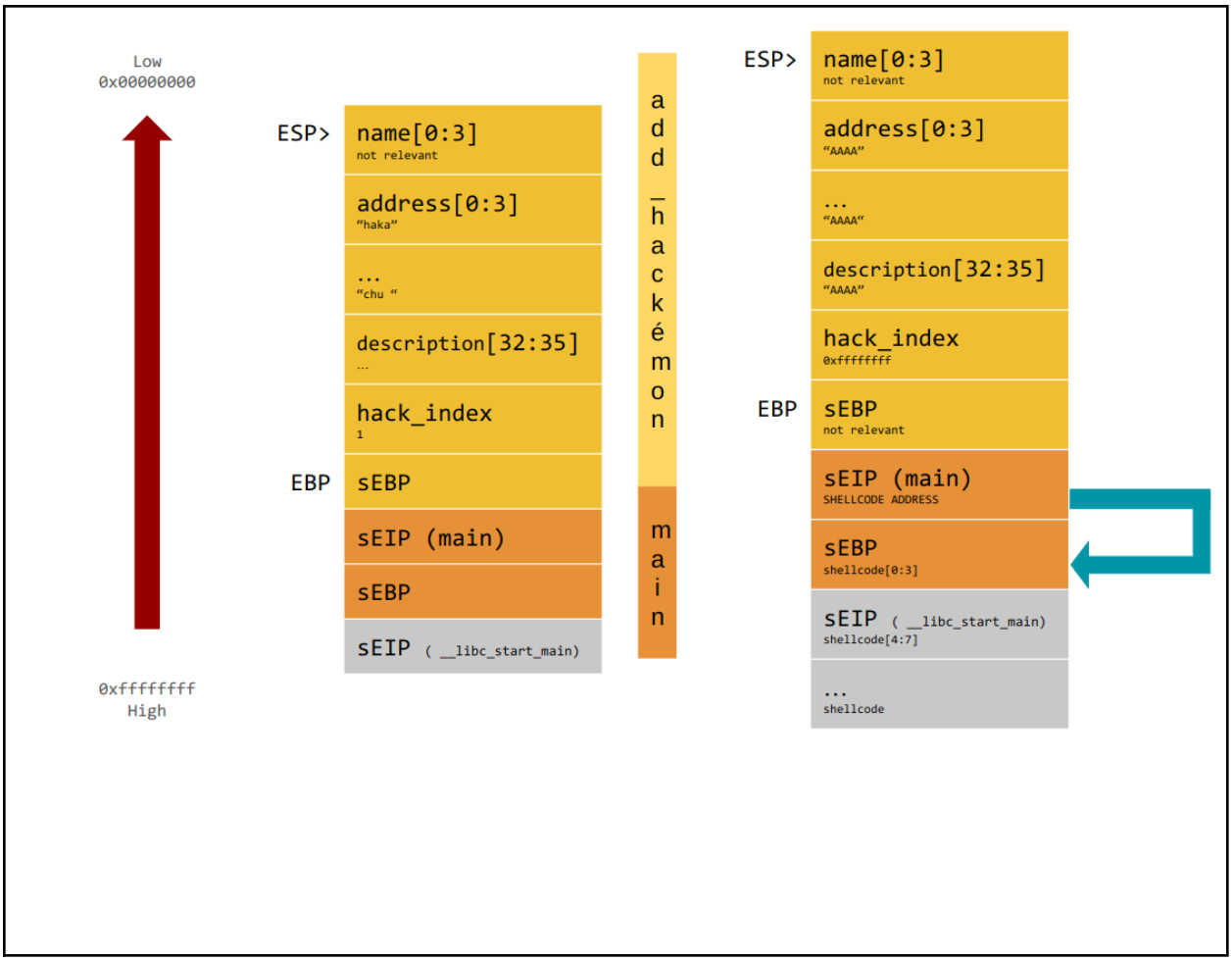
2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can exploit the buffer overflow on line 23. By overflowing the stack, you can overwrite the saved EIP (sEIP) to redirect execution to your shellcode. Due to the size of the shellcode, you need to write it after the sEIP (at higher memory locations). In other words, you will overwrite the caller's stack frame with the shellcode.

Since the counter variable that controls the while loop is between the sEIP and the h.name variable, you must overwrite the counter with a negative value to exit the while loop and trigger the return.

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- Any relevant content;
- The functions stack frames.

Show also the content of the caller frame.



[2 points] 3. To solve the problem, the senior developer, Pario Molino, decides to compile the program with a **correctly implemented** "Stack Canary" mitigation technique (i.e, randomly generated and non-guessable). Is it effective to prevent your exploit?
If it is not effective, please explain why. Otherwise, if the mitigation technique is effective, explain why and discuss whether it is possible to execute the shellcode with the format string vulnerability you identified above. If it is exploitable, write the full exploit's details, otherwise explain why it cannot be exploited.
Motivate your answer with all the necessary details.

It is effective against the previous exploit (using the BOF means overwriting the canary as well). Moreover, the canary takes space in the stack, making the BOF insufficient to write the entire shellcode.

This point admits multiple solutions. Below, a couple of possible solutions are presented. Any equivalent, working solutions are worth full grade.

Solution 1:

1. *Overwrite the* `hack_index` *value on the stack with a sufficiently high value to force the while loop to iterate many times.*
2. *At each iteration, overwrite the* `h.description` *to add a format string that will be executed when the code reaches line 26.*
3. *Use the format string to overwrite the saved sEIP and write the shellcode into the stack, one part at a time.*
4. *Overwrite the* `hack_index` *again with a negative value to exit the loop.*

Solution 2:

1. *Overwrite the* `hack_index` *value on the stack with a sufficiently high value to force the while loop to iterate many times.*
2. *Use the buffer overflow to overwrite the sEIP and write part of the shellcode.*
3. *At each iteration, overwrite the* `h.description` *to add a format string that will be executed when the code reaches line 26.*
4. *Use the format string to leak the canary and write the missing part of the shellcode.*
5. *Rewrite the correct value of the canary using the BOF.*
6. *Overwrite the* `hack_index` *again with a negative value to exit the loop.*

[1 point] 4. Driven by desperation, the senior developer, Pario Molino, decides to compile the program with **both** the "Stack Canary" and "NX" mitigation techniques.
Is it possible to get a shell using one of the techniques you know? How? Provide all the necessary details.

ret2libc / ROP [see slides]

Same trick described in 3 to overwrite the sEIP.

Exercise 4 - Web Vulnerabilities (6 points)

Welcome to WesterosBank, the modern banking solution designed to simplify your financial transactions. With WesterosBank, you can enjoy fee-free money transfers to any user at any time. Our platform ensures seamless and quick transactions without hidden charges, providing you with a streamlined and intuitive banking experience for all your financial needs.
The relevant code of the application is reported below.

```
00  @app.route('/send, methods = ['GET'])
01  def send(request, session):
02      if not is_session_valid(session):
03          return redirect(url_for("/login"))
04      sender_id = session["user_id"]
05      if "receiver" not in request.keys():
06          return render_template("home.html", error="No receiver specified!")
07      receiver = request["receiver"]
08      if "amount" not in request.keys():
09          return render_template("home.html", error="No amount specified!")
10      amount = int(request["amount"])
11      if amount < 0:
12          return render_template("home.html", error="Invalid amount")
13      check_b = "SELECT balance FROM users where user_id = " + str(sender_id) + ";"
14      result = db_exequte_query(check_b)
15      balance = result[0]["balance"]
16      if amount > balance:
17          return render_template("home.html", error="Invalid amount")
18      check_r = "SELECT user_id FROM users WHERE username = '" + receiver + "';"
19      result = db_exequte_query(check_q)
20      if not result:
21          return render_template("home.html", error="Receiver" + xss_sanitize(receiver)  +
              " not found!")
22      receiver_id = result[0]["user_id"]
23      update_q = "UPDATE users SET balance = balance + amount WHERE user_id = " +
              str(receiver_id) + ";"
24      db_exequte_query(update_q)
25      update_q = "UPDATE users SET balance = balance - amount WHERE user_id = " +
              str(sender_id) + ";"
26      db_exequte_query(update_q)
27      return render_template("home.html", success="Money transferred!")
28
29  @app.route('/login, methods = ['POST'])
30  def login(request, session):
31      if "username" not in request.keys():
32          return render_template("login.html", error="No username specified!")
33      username = request["username"]
34      if "password" not in request.keys():
35          return render_template("login.html", error="No password specified!")
36      password = request["password"]
37      query = "SELECT user_id FROM users WHERE username = ? AND password = ?;"
38      result = db_execute_query_with_statement(query, username, password)
39      if len(result) != 1:
40          return render_template("home.html", error="Invalid username or password")
41      session["user_id"] = int(result[0]["user_id"])
42      return url_for("home.html")
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)	balance (int)

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** executes queries with prepared statements.
- The function **db_execute_query** executes queries without prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT …; INSERT INTO …;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.
- The function **xss_sanitize** correctly escapes all the HTML entities, i.e., all the HTML special characters are converted to their equivalent data (> becomes >).

More details:

- The function **render_template** generates an HTML page from a .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL of the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The functions **db_execute_query_with_statement** and **db_execute_query** return a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
SQL Injection (SQL-I)	*Line 18. An attacker can read data from the DB with an injection on the variable "receiver".*	*Using prepared statements.*

cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*Not present.*	*Nothing to do.*
Cross-site request forgery (CSRF)	*CSRF is not implemented for the function "send". An attacker can potentially transfer money on another behalf*	*Implement a CSRF token correctly randomized for the function "send"*

[1.5 points] 2. After visiting a link on 4chan.com, you notice that your balance has decreased by 1000$ since you sent that amount to the user *Law*. However, you clearly didn't want to send such an amount to Law. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The website contains a script that visits the URL:

https://westerosbank.com/send?balance=1000&receiver=Law

NB: it's a get request so there's no need for a form and we can put all the parameters in the URL

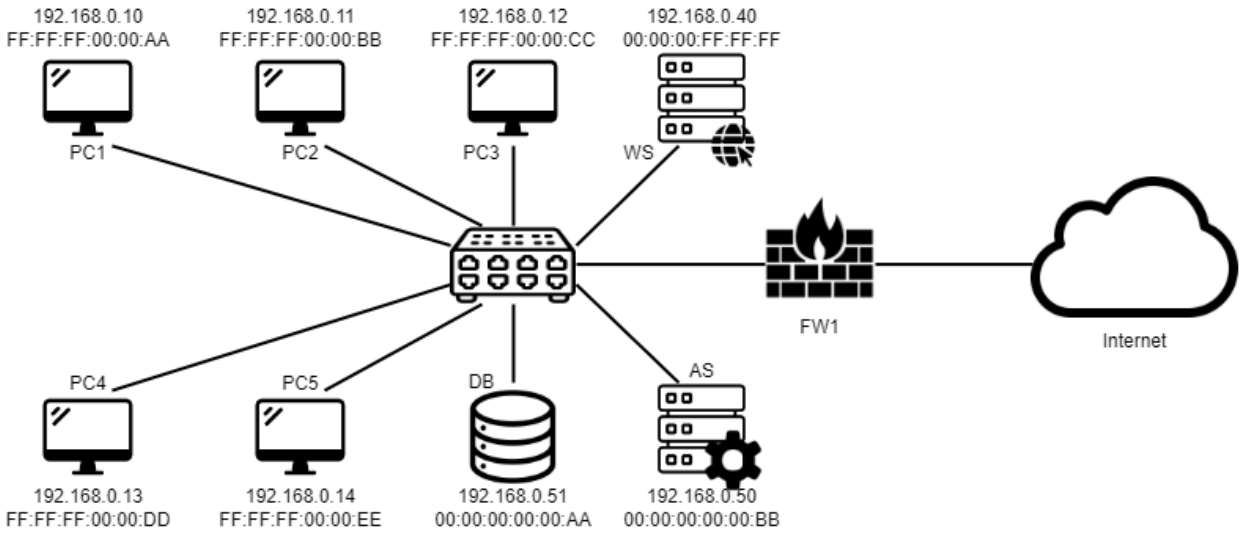
[1.5 points] 2. You suddenly notice that somebody transferred all the money to a user. You suspect that somebody logged in with your profile even if your password is secure. Can you explain how somebody could have obtained your password? Explain all the steps you would take to reproduce the exploit, including at least a sketch of the final code you would write.

Blind SQLInjection. The attacker exploited the SQL injection in the variable "receiver" to exfiltrate the password of the user.

receiver = "aaaaaaaaaaaaaa' OR username='<victim_username>' AND password LIKE 'a%"

This will return 1 row only if the password of the victim starts with a. If it doesn't try with b and so on. Repeat the process for all the characters of the password.

Exercise 5 - Network Security (6 points)



SharedCars is a new car-sharing company that opened its first office in Milan. The office is relatively small: there are four workstations for the employees and one for the CEO. SharedCars developed an application that users can download on their smartphones, from which they can register to insert all their data: name, surname, date of birth, driving license number and photo, and billing information. They also developed a website that can be used to perform the same operations.
Currently, SharedCars only uses a few servers to provide its services to users and intends to expand soon. In particular, they have a Web Server (that hosts the website), an Application Server (that hosts the logic of the application), and a Database Server (which hosts their database).

- The Web Server is accessible over HTTP(S), and needs to access the Application Server.
- The smartphone app makes API requests to the Web Server over HTTP(S).
- The Database Server must not be accessible from the outside. Only the Application Server (and the employees) can access it.

Shared Cars also installed a firewall and hired you as an external consultant to configure it. The network layout is represented in the figure above.

1. [1 points] Write the firewall rules, assuming the firewall to be a **stateful packet filter**.

Firewall	Src IP (or entity names)	Src PORT	Direction of the 1st packet	Dst IP (or entity names)	Dst PORT	Policy	Description
FW1 (example)	*10.0.0.1 (example)*	*ANY*	*zone 1 -> zone 2*	*192.168.0.2 (example)*	*443*	*ALLOW*	*(example: the X server in zone 1 cannot contact the Y server)*
FW1	ALL	ANY	ALL	ALL	ANY	DENY	Default deny
FW1	ANY	ANY	INT->LOCAL	WS_IP	80/443	ALLOW	Access for users to the WS
FW1	PC_IP	ANY	LOCAL->INT	ANY	ANY	ALLOW	Internet access for the employees

2. [2 points] SharedCars is under attack. Users noticed their data was stolen after registering for this service, and the company called you to investigate what was happening. You open your network analysis tools and find this sequence of packages captured by PC1:

```
|     SrcIP        |        SrcMAC       |       DestIP      |        DestMAC      |  Type          |
|------------------|-------------------|------------------|-------------------|---------------|
| 38.133.177.84    | 5f:f7:b9:17:e0:d1 | 186.91.69.145     | 04:14:33:1b:49:74 |  SIP           |
| 197.143.52.253   | 04:14:33:1b:49:74 | 206.242.18.228    | 36:d7:a0:55:11:9d |  HTTP          |
| 190.189.171.107  | ea:84:5d:aa:cf:03 | 188.23.77.152     | a7:5a:3b:55:39:37 |  DNS           |
| 133.173.40.253   | 35:1f:0e:05:9a:b9 | 60.183.135.241    | da:be:85:6b:77:72 |  TCP           |
| 65.167.63.6      | 4a:c5:59:a3:d0:4e | 54.100.134.150    | 2b:af:3f:3d:2b:bf |  UDP           |
[...]
| 111.145.38.202   | f5:0b:58:78:07:d4 | 213.226.165.24    | 9c:81:e7:13:e5:91 |  ICMP          |
| 192.168.0.51     | 00:00:00:00:00:AA | 255.255.255.255   | ff:ff:ff:ff:ff:ff |  ARP request   |
| 156.234.51.34    | ff:ff:ff:ff:ff:ff | 255.255.255.255   | ff:ff:ff:ff:ff:ff |  ARP response  |
| 198.65.45.154    | ff:ff:ff:ff:ff:ff | 255.255.255.255   | ff:ff:ff:ff:ff:ff |  ARP response  |
| 187.98.65.145    | ff:ff:ff:ff:ff:ff | 255.255.255.255   | ff:ff:ff:ff:ff:ff |  ARP response  |
| 245.99.182.158   | 7d:90:1d:64:0e:6f | 60.196.10.56      | 92:99:ce:29:cf:d0 |  FTP           |
| 216.75.102.35    | 9f:23:c0:af:85:5f | 39.124.31.135     | 92:73:e8:6b:df:fc |  DHCP          |
```

You check the traffic visible from the other PCs and find the same sequence of packets. Going through the logs of the switch, you notice that these packets come from the port associated to the web server. You also notice this particular packet sequence:

```
|     SrcIP      |        SrcMAC       |      DestIP      |        DestMAC       |  Type   | Sw. slot |
|---------------|--------------------|----------------|--------------------|--------|----------|
| 192.168.0.50  | 00:00:00:FF:FF:FF | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:FF:FF:FF  |SQLresp.|    DB    |
| 192.168.0.50  | 00:00:00:FF:FF:FF | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:FF:FF:FF  |SQLresp.|    DB    |
| 192.168.0.50  | 00:00:00:FF:FF:FF | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:FF:FF:FF  |SQLresp.|    DB    |
```

What attack(s) is(are) going on? Who is the attacker? What is their goal?

There is a CAM Flooding attack and an ARP spoofing attack.
The attacker is someone who controls the Web Server.
From the last sequence of packets, it seems that the attacker's goal is to retrieve data from the DB.

3. [1 point] Can you mitigate the attack(s) by acting on the network switch only? If yes, how?

CISCO Port Security or a similar mechanism.

4. [2 points] SharedCars asks you to solve the problem. You can modify the network layout as you like and add the necessary firewall rules. Draw the network **in a clear way**, justify your answer, and write the new firewall rules in the table below.

You should add a DMZ, dividing the web server from the rest of the network.
As a best practice, you should place employee PCs and servers in two separate subnetworks.

Firewall	Src IP (or entity names)	Src PORT	Direction of the 1st packet	Dst IP (or entity names)	Dst PORT	Policy	Description

Exercise 6 - Malware (6 points)

Neo Tokyo 3 is a town in Japan equipped with advanced defense technologies to protect its civilians from attacks by the Angels, a group of mysterious and dangerous creatures threatening humanity. The defense is managed by the NERV agency, which relies on the Magi system to support its operations. Unfortunately, a sophisticated Angel attack commenced by infecting the Magi system with malware. After successfully defeating the Angel threat, Gendo Ikari, the commander of NERV, has sent you a sample of the malware for analysis.

```
01    int main() {
02          int ecx;
03          __asm__ (
04                "mov eax, 1"  // Sets the parameter of the CPUID instruction
05                "cpuid"       // Execute the CPUID instruction
06                : "=c"(ecx)   // Stores the result in the ecx variable
07          );
08
09          if ((ecx >> 31) & 0x1) // Checks if the 31th bit of ecx is 0x1
10                exit();
11          else
12                malicious_stuff(); // Call the malicious code
13    }
```

Here some useful informations to analyze the malware:

- the *cpuid* instruction retrieves, on X86 systems, information about the CPU. The retrieved information depends on the parameter passed to the cpuid instruction though the eax register.
- when eax is set to 0x1, cpuid queries information about the features of the processor, and returns them inside the ecx register.
- according to Intel's X86 manual, the 31th bit of the value returned in the ecx register is always zero. However, certain virtualization environments, e.g. Microsoft's HyperV, sets this bit to 1 to mark that code is running inside a virtualized environment.

[2 points] 1. This malware employs **one** evasive technique? Please explicitly state which evasive technique is being employed, and explain how it works in this particular case. Please specify all the assumptions you have made. Please note: giving vague or imprecise answers, or naming wrong techniques will make you lose points.

The malware is implementing anti-virtualization. The call to cpuid retrieves info about the processor, and in particular the code is checking if the hypervisor bit is set to 1.

[2 points] 2. Which type of analysis would you apply to the collected sample? Please state also **why it is applicable, and how you would conduct such an analysis**. Answers without such information will be considered incomplete. Please specify all the assumptions you have made.

You can do both static and dynamic analyses, under different assumptions (one of the following combination of answer is enough):

- Static analysis can always be done. The malware is nor metamorphic or polymorphic, the code isn't encrypted, and it's possible to both decompile it, or use a signature detection based tool.
- Dynamic analysis: in practice, you have several options to conduct it on a sandbox:
  - Patch the binary to skip the hypervisor bit check
  - Use a custom sandbox, which doesn't set to 1 hypervisor bit
  - Use a bare metal OS on a isolated PC to run behavioral analysis (or a debugger).

While analyzing the infection on the Magi system, Gendo Ikari found another variant of the malware.

```
int main() {
       int ecx;

       __asm__ (
           "mov eax, 1"
           "cpuid"
           : "=c"(ecx)
           );
```

```
int main() {
       int ecx;

       __asm__ (
           "push ecx"
           "xor ecx, ecx"
           "add ecx, 1"
           "nop"
           "nop"
           "mov eax, ecx"
           "pop ecx"
           "cpuid"
           : "=c"(ecx)
```

`    if ((ecx >> 31) & 0x1)`
`        exit();`
`    else`
`        malicious_stuff();`
`}`
	`        );`
	
	`    if ((ecx >> 31) & 0x1)`
	`        exit();`
	`    else`
	`        malicious_stuff();`
	`}`

Old Example	New Example

3) [2 points] Compared to the original version, what new stealth technique does this malware implement? Which type of analysis would you apply to the new sample? Please state also **why it is applicable, and how you would conduct such an analysis**. Answers without such information will be considered incomplete. Please specify all the assumptions you have made. Please note: giving vague or imprecise answers, or naming wrong techniques will make you lose points.

New Example employs metamorphism. In fact, there are some useless instruction to set eax to 1.
You can conduct Static Analysis only doing code decompilation. In fact, signature detection will fail since it is different from the one collected from the first sample.
You can use dynamic analysis as well, with the same conditions as stated above.